Software Requirements Specification

Use Case Based Capstone Template

Document field	Team entry
Project name	‘Royal Budget’; A Personal Finance & Budgeting Assistant
Team name and members	RoyalDev - Cesar Giner, Jacob Melookaran, Joshua Webber, Jake Hall
Sponsor and instructor	Commerce Bank mentors and Shah, Syed Jawad Hussain
Version and status	V1.0, Updated Requirements Specification Document
Date	November 9, 2026
Requirements lead	Cesar Giner

Revision history

Version	Date	Author	Change and approval
0.1	October 2, 2026	RoyalDev Team	Initial actor list, use cases, and catalog
1.0	November 9, 2026	Cesar Giner	Updated many attributes of the requirements specification document to better reflect the progress and state of the Budget Application.

Contents

1 Introduction

2 Stakeholders Actors and Goals

3 Product Context and Constraints

4 Use Case Model

5 Functional Requirements and Detailed Use Cases

6 Nonfunctional Requirements

7 Data and External Interfaces

8 Traceability and Sprint Mapping

9 Review and Change Control

Appendix A Reusable Detailed Use Case Form

1 Introduction

1.1 Purpose and audience

This document provides the formal Software Requirements Specification, detailed use case models, functional requirements, and non-functional specifications for the Commerce Bank Personal Finance & Budgeting Assistant web application. It acts as the definitive technical baseline for software architecture design, sprint breakdown, database modeling, unit and integration testing, and formal stakeholder evaluations.

The target audience includes project sponsors from Commerce Bank, technical course instructors, capstone evaluators, and the RoyalDev engineering team.

1.2 Problem and product vision

Modern consumers, particularly the younger demographic, face significant challenges in building financial literacy, organizing personal budgets, and tracking their spending.

The Commerce Bank Personal Finance & Budgeting Assistant delivers an intuitive full-stack web application designed to bridge this gap. The platform provides monthly budget planning, the ability to record and track income and expenses, and an AI assistant that shares contextual insights tailored to each user. These services are designed to empower users to take control of their financial wellness in a risk-free environment.

1.3 Scope

In scope	Out of scope
User account registration, secure login, secure session management, and user profile and settings.	Direct connections to live banking APIs
MySQL relational database backend and a responsive React web interface styled with Tailwind CSS.	Real credit card processing
Monthly budget creation with a selectable plan, category allocations, and plan-vs-actual progress tracking.	Real monetary transfers, payment processing networks, or automated clearing house (ACH) operations.
External AI/LLM integration to deliver customized financial advice based on anonymized spending metrics.	Storing sensitive information, ie payment card numbers or real bank credentials.
Income and expense transaction logging using fabricated data, organized by month.	

1.4 Definitions and conventions

Term	Definition
Actor	A person, role, device, or external system outside the product boundary that interacts with the product.

Term	Definition
Use case	A goal-oriented interaction between an actor and the system.
Scenario	One path through a use case, including the main success path or an alternate or exception path.
Functional requirement	A capability, rule, validation, calculation, or system response that the product must provide.
Nonfunctional requirement	A measurable quality attribute or constraint on how the product operates or is built.
Must	Mandatory requirement.
Should	Desired requirement that may be deferred with an approved reason.
Could	Optional requirement, delivered only if time allows.
Budget plan	A preset split of the monthly budget into needs, wants, and savings: Balanced (50/30/20), Save more (40/20/40), or Essentials (60/20/20).
Allocation	The expected spending a user assigns to one need or want category within a monthly budget.
Session	The signed-in state of a user, kept by a secure cookie that expires after 12 hours or on logout.

1.5 Assumptions dependencies and open questions

ID	Type	Statement	Owner	Target resolution
A-001	Assumption	Users possess modern web browsers and basic internet connectivity.	RoyalDev Team	Ongoing; checked during usability testing
D-001	Dependency	External AI/LLM API service availability	Cesar Giner	Before AI assistant development
Q-001	Open question	Which specific LLM provider and model version should be targeted for budget insights?	Commerce Bank Mentors/ Instructor	Before AI assistant development
A-002	Assumption	Database hosting and environment setup will be available early in the project	Jacob Melookaran	Before Back-end Logic implementation (Resolved)
Q-003	Open question	Do the OWASP ZAP security scan and file logging satisfy Commerce Bank's two required stretch goals?	Team	Before Final Presentation

2 Stakeholders Actors and Goals

Identify people and external systems before naming use cases. An actor represents a role, not a specific teammate or screen.

2.1 Stakeholders

Stakeholder	Interest or decision	How requirements will be validated
Commerce Bank mentors	Sponsor alignment and technical feedback	Midsemester review, questions, and final walkthrough
Instructor	Course expectations and evaluation	Scheduled reviews and submission feedback
RoyalDev Team	Project development	Tasks and deliverables will be validated with team oversight, internal testing and feedback from course instructor
Target users / testers	An intuitive budgeting tool usable regardless of technical background	Informal usability sessions completing key tasks (register, create a budget, record transactions, etc)

2.2 Actors and user characteristics

Actor ID	Actor or role	Goals	Knowledge permissions and needs
ACT-01	Registered User	Manage monthly category budgets, simulated transactions, track financial goals, and receive budgeting advice.	Standard web user with basic digital experience. Requires authenticated access to personal financial data and an intuitive responsive web interface.
ACT-02	AI/LLM Service	Ingest anonymized spending metrics and return contextual, actionable personal finance recommendations.	External system integration. Must only receive anonymized context without real financial data.

2.3 Actor goal summary

Actor	Goal	Success outcome	Candidate use case
Registered User	Authenticate and manage personal account details	User account is established and an authenticated session is started	UC-AUTH-01, UC-AUTH-02, UC-AUTH-03, UC-PROF-01
Registered User	Create a monthly budget with a plan and category allocations	The budget is stored and displayed with plan-vs-actual progress	UC-BUD-01
Registered User	Log income and expense transactions	The transaction is validated, saved, and plan-vs-actual progress updates	UC-FIN-01
AI/LLM Service	Generate contextual personal finance advice	Actionable financial insights are safely rendered for the user	UC-AI-01

3 Product Context and Constraints

3.1 Product environment and system boundary

The system architecture follows a full-stack web application model consisting of three primary layers. The Frontend Presentation Layer is built using React styled with tailwind to deliver a fully responsive interface. The Backend Application Server built using Python with FastAPI and SQLAlchemy, operates as a server responsible for executing core business and application logic, routing, and security filters. Finally, the Database Layer utilizes a MySQL relational database server to enforce strict data integrity.

3.2 Sponsor brief baseline

- Build a full-stack web application selected by the team; do not use a CMS such as WordPress.
- Include a meaningful financial component such as budget tracking, simulated payments, or investments.
- Support user registration and login; store passwords as secure hashes, never as plain text.
- Use a database, a front-end framework or library, and one CSS framework.
- Provide Login/Register, Home/Dashboard, User Profile/Settings, and at least two additional pages.
- Use only fake or dummy card and financial data when simulating transactions.
- Include unit testing; automated tests are strongly encouraged.

3.3 Chosen stretch goals

Stretch goal	Reason for selection	Use cases or requirements affected	Evidence
Security scan (OWASP ZAP)	Finds and fixes vulnerabilities in an application that handles personal financial data, and produces a scan report for the final presentation.	All use cases; NFR-SEC-003	ZAP report before and after fixes, showing 0 crit findings
Data Visualization	Provides users with intuitive, interactive visual charts to track spending trends over time and gain deeper financial insights.	UC-BUD-01	Interactive dashboard demo showing monthly category breakdown and trend analysis charts.

3.4 Business rules

Rule ID	Rule	Source	Related use cases
BR-001	Development, testing, and demonstrations must not use sensitive information	Sponsor brief	UC-FIN-01, UC-AI-01
BR-002	User passwords must be securely hashed using salted algorithms prior to persistence.	Security policy	UC-AUTH-01
BR-003	The solution must be built as a custom full-stack web application; off-the-shelf website builders or CMS platforms are not permitted.	Course guidelines	All use cases
BR-004	Prompts sent to external AI services must strip out all private information and transmit only relative, anonymized spending and category metrics.	Privacy Policy	UC-AI-01

Rule ID	Rule	Source	Related use cases
BR-005	Users can view and change only their own data; the user is always identified from the session, never from request data.	Security policy	UC-BUD-01, UC-FIN-01, UC-PROF-01
BR-006	A user may have only one budget per calendar month. Plan percentages are set by the server from the selected plan, not by the client.	Team design decision	UC-BUD-01
BR-007	A transaction's category must match its type: income uses income categories; expenses use need or want categories.	Team design decision	UC-FIN-01
BR-008	Budget warnings (over a target, below the savings target, or over income) inform the user but never block saving.	Team design decision	UC-BUD-01

4 Use Case Model

4.1 Use case diagram

External Actors (Left)	System Boundary: Application System	External Actors (Right)
ACT-01: Registered User *(Primary User)*	• UC-AUTH-01: Register an account • UC-AUTH-02: Sign in • UC-AUTH-03: Persistent User Sessions • UC-AUTH-04: Sign Out • UC-BUD-01: Create a monthly budget • UC-BUD-02: Budget Progress • UC-FIN-01: Log simulated transaction • UC-FIN: Transaction history • UC-AI-01: Generate AI budget insights • UC-PROF-01: Display Profile Info and Settings	ACT-02: AI/LLM Service *(External System)*

NOTE: 'UseCase' Document in Github repo '/docs' folder for a more detailed information for each use case.

4.2 Use case catalog

ID	Use case name	Primary actor	Priority	Status	Target
UC-AUTH-01	Register an account	Registered user	Must	Implemented	Sprint 1
UC-AUTH-02	Sign in	Registered user	Must	Implemented	Sprint 1
UC-AUTH-03	Persistent User Sessions	Registered user	Must	Implemented	Sprint 2
UC-AUTH-04	Sign Out	Registered user	Must	Implemented	Sprint 2
UC-BUD-01	Create a monthly budget	Registered user	Must	Implemented	Sprint 2
UC-BUD-02	Budget Progress	Registered user	Must	Implemented	Sprint 2
UC-DASH-01	View Dashboard	Registered user	Must	Implemented	Sprint 3
UC-FIN-01	Log simulated transaction	Registered user	Must	Implemented	Sprint 3
UC-FIN-02	View Transaction History	Registered user	Must	Implemented	Sprint 3
UC-AI-01	Generate AI budget insights	Registered user (AI/LLM Service supporting)	Must	Planned	Sprint 4

ID	Use case name	Primary actor	Priority	Status	Target
UC-PROF-01	Display Profile Info and Settings	Registered user	Must	Planned	Sprint 4

4.3 Use case relationships and scope notes

Relationship or boundary	Explanation and reason
UC-AUTH-02 includes UC-AUTH-03	Every successful sign-in creates a session. All other user use cases require an active session as a precondition.
UC-AI-01 depends on UC-BUD-01 and UC-FIN-01	AI insights are generated only from the signed-in user's own budget and transaction summaries. Without a budget, insights are based on transactions only.

5 Functional Requirements and Detailed Use Cases

Use cases are the primary source of functional requirements. Copy the reusable form in Appendix A for every high-priority use case. Complete the main success scenario and realistic alternate and exception flows before the related story enters a sprint. Add a separate FR statement when behavior applies across use cases or needs more precision than a flow step provides.

5.1 Functional requirements catalog

Use this catalog for required behavior that applies across use cases or needs a precise standalone statement. Sponsor-brief capabilities such as required pages belong here, not in the nonfunctional section. Every functional requirement must link to at least one use case, business rule, or sponsor constraint and must be testable.

FR ID	Functional requirement	Source or related use cases(UseCase ID)	Priority	Verification
FR-AUTH-01	The system must let a prospective user register with a unique username and email, a name, and a password of 8 to 72 characters; duplicate usernames or emails are rejected.	UC-AUTH-01	Must	TC-AUTH-01
FR-AUTH-02	The system must let a registered user sign in with a username and password, returning the same generic error for an unknown username or a wrong password.	UC-AUTH-02	Must	TC-AUTH-02
FR-AUTH-03	After sign-in, the system must keep the user authenticated across page refreshes with a secure session cookie that expires after 12 hours.	UC-AUTH-03	Must	TC-AUTH-03
FR-AUTH-04	The system must let a signed-in user sign out, which revokes the session on the server and clears the session cookie.	UC-AUTH-04	Must	TC-AUTH-04
FR-BUD-01	The system must let a user create one budget per calendar month by entering an expected amount (whole dollars, $1 to $1,000,000,000) and choosing a plan (Balanced, Save more, or Essentials).	UC-BUD-01	Must	TC-BUD-01

FR ID	Functional requirement	Source or related use cases(UseCase ID)	Priority	Verification
FR-BUD-02	The system must let the user allocate expected spending to each need and want category; categories left empty are saved as $0.	UC-BUD-01	Must	TC-BUD-02
FR-BUD-03	The system must show the needs, wants, and savings targets and warn, without blocking saving, when allocations exceed a target or the expected budget.	UC-BUD-01	Must	TC-BUD-03
FR-BUD-04	The system must show the selected month's plan vs. actual: spending per need and want category, income received, and projected savings.	UC-BUD-02	Must	TC-BUD-04
FR-FIN-01	The system must let a user record income and expense transactions with an amount (up to 2 decimals), a category matching the type, a date, and optional notes.	UC-FIN-01	Must	TC-FIN-01
FR-FIN-02	The system must list a user's transactions by month, newest first, with monthly income and expense totals.	UC-FIN-02	Must	TC-FIN-02
FR-DASH-01	The system must show a dashboard with the current month's projected savings, needs and wants progress, spending so far, and the 5 most recent transactions.	UC-DASH-01	Must	TC-DASH-01
FR-AI-01	The system must let a signed-in user ask an AI assistant budgeting questions and receive answers based on a summary of their own budget and transactions.	UC-AI-01, UC-BUD-01, UC-FIN-01	Must	TC-AI-01
FR-PROF-01	The system must provide a Profile and Settings page that displays the signed-in user's username, name, email, and account creation date.	UC-PROF-01	Must	TC-PROF-01
FR-PROF-02	The system should let the user update their name and email and change their password (current password required).	UC-PROF-01	Should	TC-PROF-02

6 Nonfunctional Requirements

State measurable quality attributes and constraints that apply across the product or to named use cases. Every NFR must include operating conditions, a target or threshold, and a verification method. Do not write vague statements such as the system must be fast or secure.

ID	Category	Requirement and measure	Verification	Priority
NFR-SEC-001	Security	Passwords must be stored only as salted bcrypt hashes; plain-text passwords must not appear in storage or logs.	Integration inspection and log review	Must
NFR-PRIV-001	Privacy	The application and all demonstrations must use protect sensitive information	Repository database and demo review	Must
NFR-USE-001	Usability	At least 80% of first-time test users can register, create a monthly budget, and record a transaction without assistance.	Usability test with representative users	Must

ID	Category	Requirement and measure	Verification	Priority
NFR-PERF-001	Performance	Budget, transaction, and dashboard API requests respond within seconds for a user with up to 1,000 transactions, measured on a local development machine.	Timed API test	Should
NFR-ACC-001	Accessibility	Text remains readable and pages remain usable when the browser zoom is set to 200%.	Manual test at 200% zoom	Should
NFR-REL-001	Reliability	If the server or database is unavailable, pages show a clear error message instead of failing silently, and a budget is saved together with all its allocations or not at all.	Fault test: stop the backend or database during use	Must
NFR-MNT-001	Maintainability	All tests pass and backend code coverage is at least 70% before final delivery; database credentials and API keys are kept in environment files(.env).	Test and coverage reports; code review	Should
NFR-SEC-002	Security	An OWASP ZAP scan shows 0 critical or high findings before final delivery; errors and failed logins are logged to a file without passwords or tokens.	ZAP report and log review	Must

NFR ID	Quality requirement and measure	Related use cases	Verification
NFR-UI-001	For the selected representative workflows, all interactive controls must be reachable and operable by keyboard.	UC-AUTH-01, UC-AUTH-02, UC-BUD-01, UC-FIN-01	Keyboard walkthrough
NFR-UI-002	Labels, validation messages, and navigation must be understandable to representative users completing the selected tasks.	UC-AUTH-01, UC-AUTH-02, UC-BUD-01, UC-FIN-01, UC-PROF-01	Usability test with stated success criterion
NFR-UI-003	Pages remain usable on screens without horizontal page scrolling.	All user use cases	Browser device-toolbar test at phone widths

7 Data and External Interfaces

7.1 Data requirements

Data ID	Entity or data	Important fields and validation	Ownership privacy and retention	Related use cases
DATA-USER	User account	id (PK); username (3 to 50 characters, letters, numbers, underscore; unique); name (1 to 100); email (valid; unique); password_hash (bcrypt); created_at; updated_at	Owned by the user. The password is stored only as a hash. Deleting a user deletes their sessions, budgets, and transactions.	UC-AUTH-01, UC-AUTH-02, UC-PROF-01
DATA-SESSION	Session	id (PK); user_id (FK); token_hash (SHA-256, 64 characters, unique); expires_at; created_at; is_revoked	The raw token exists only in the HttpOnly cookie and is never stored. Sessions expire after 12 hours and are revoked on logout.	UC-AUTH-02, UC-AUTH-03, UC-AUTH-04
DATA-CATEGORY	Category	id (PK); name (unique); kind (income, need, or want)	Shared reference data: 14 seeded categories, read-only for users.	UC-BUD-01, UC-BUD-02, UC-FIN-01, UC-FIN-02

Data ID	Entity or data	Important fields and validation	Ownership privacy and retention	Related use cases
DATA-BUDGET	Monthly budget	id (PK); user_id (FK); budget_month (first day of the month; unique per user); expected_amount (DECIMAL 12,2, at least 0); needs_pct, wants_pct, savings_pct (sum to 100)	Owned by the user; deleted id the user is also deleted.	UC-BUD-01, UC-BUD-02, UC-DASH-01
DATA-ALLOCATION	Budget allocation	id (PK); budget_id (FK); category_id (FK); amount (at least 0); unique per budget and category	Belongs to a budget; deleted with the budget.	UC-BUD-01, UC-BUD-02
DATA-TRANSACTION	Transaction	id (PK); user_id (FK); category_id (FK, nullable); amount (greater than 0, 2 decimals); transaction_type (income or expense); transaction_date; description (optional, up to 255 characters)	Owned by the user; fabricated data only. Kept until the user is deleted.	UC-FIN-01, UC-FIN-02, UC-BUD-02, UC-DASH-01

7.2 External interfaces

Interface ID	System or service	Purpose and contract	Authentication data and failure behavior	Owner
INT-001	REST API (FastAPI)	JSON over HTTP(S) under /api: 10 endpoints for authentication, categories, budgets, and transactions (	Session cookie authentication; 401 when not signed in, 422 for invalid input; errors returned as {"detail": ...}.	Cesar Giner
INT-002	External AI/LLM API (provider to be decided, Q-001)	The backend sends a system prompt, a summary of the user's budget and transactions, and the user's question; it receives a text answer.	API key stored only in the backend environment. On timeout or failure, the assistant shows an "unavailable" message. Provider spending limit and per-user message limit.	Cesar Giner

8 Traceability and Sprint Mapping

8.1 Sprint backlog mapping

Jira Framework for Project Sprint Mapping: https://umkc-team-hlh86r9q.atlassian.net/jira/software/projects/CP/list?jql=project%20%3D%20CP%20ORDER%20BY%20cf%5B10019%5D%20ASC

8.2 Prioritization

Must requirements, including all sponsor-mandated pages and unit testing, are completed before Should and Could work. Sprint order followed dependencies: authentication and sessions first, since every other use case requires a signed-in user, then budgets, then transactions, then the dashboard that combines them. Remaining Must work (Profile and Settings, testing, the AI assistant, and the two stretch goals) precedes Should items such as data visualization. If time runs short, Could items are cut first, then Should; Must items and security standards are not cut.

9 Review and Change Control

9.1 Review checklist

☑ Every actor has at least one goal and every in-scope goal maps to a use case.

☑ The use cases describe behavior and outcomes without unnecessary screen or database design.

☑ Functional requirements describe testable behavior and use stable FR identifiers.

☑ Nonfunctional requirements include measurable targets and verification methods.

☐ Use cases, FRs, and NFRs link to sprint stories and test cases by stable identifier. (Test cases pending.)

☑ All examples, fixtures, screenshots, and demonstrations use fabricated financial data.

9.2 Change requests and decisions

Change ID	Requested change and reason	Affected IDs	Date sprint
CHG-001	Drop stored payment methods; users record transactions manually, so no card data is stored.	UC-FIN-01, BR-001, DATA-TRANSACTION	Oct 2026
CHG-002	Change the stack from Bootstrap and PostgreSQL to Tailwind CSS and MySQL.	1.3, 3.1	Oct 2026
CHG-003	Replace the stretch goals (data visualization, persistent sessions) with an OWASP ZAP security scan and file logging. Basic sessions remain a core requirement; data visualization becomes a Should feature.	3.3, NFR-SEC-003, Q-003	Nov 2026
CHG-004	Set the session lifetime to 12 hours following NIST SP 800-63B AAL2; no long-lived sessions for financial data.	FR-AUTH-03, NFR-SEC-002	Oct 30, 2026
CHG-005	Remove the System Administrator and Guest actors; admin features are out of scope for the capstone.	2.2, 4.2, FR-ADM-01, UC-ADM-01	Nov 2026
CHG-006	Add use cases for sign out, budget progress, the dashboard, and transaction history, with matching FRs and data links.	UC-AUTH-04, UC-BUD-02, UC-DASH-01, UC-FIN-02, FR-AUTH-04, FR-BUD-04	Nov 2026